

# TerpAI scheduling client — handoff guide

ShellAI / UMD integration

May 2026

## Contents

|                                                   |   |
|---------------------------------------------------|---|
| 1 Purpose and audience                            | 1 |
| 2 Scope and limitations                           | 1 |
| 3 Prerequisites                                   | 2 |
| 4 Environment variables                           | 2 |
| 5 Data contract — <code>jupitersp_response</code> | 2 |
| 6 Public API (summary)                            | 3 |
| 7 How to run locally                              | 3 |
| 8 Operational flow (text)                         | 4 |
| 9 Security and compliance                         | 4 |
| 10 Troubleshooting                                | 4 |
| 11 Related files in this repository               | 5 |
| 12 Revision history                               | 5 |

## 1 Purpose and audience

This document explains the Python module `backend/terpai_client.py`, which calls the UMD TerpAI (Nebula) **internal segments** HTTP API to produce a short **scheduling-oriented plain-text summary** from a Jupiterp-style course payload.

**Audience:** engineers who need to configure credentials, run a local smoke test, or embed `compute_terpai_scheduling_summary` next to existing course APIs.

## 2 Scope and limitations

- **Bearer JWTs expire**; there is no refresh flow in this module. Operators must rotate `TERPAI_BEARER` manually until a supported server-side token exchange exists.

- This client uses the **same streaming data: line protocol** as `terpai_test.py` / `terpai_chat.py`; TLS trust and institutional CA issues are environment-specific (see project `README.md` for proxy alternatives under `/api/terpai`).

### 3 Prerequisites

| Requirement  | Notes                                                                                                                                                                                 |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python       | 3.10+ recommended (uses modern optional type syntax).                                                                                                                                 |
| Dependencies | <code>requests</code> (listed in <code>backend/requirements.txt</code> ).<br>Optional: <code>python-dotenv</code> to load <code>.env</code> / <code>.env.terpai</code> automatically. |
| Network      | Outbound HTTPS to <code>TERPAI_BASE_URL</code> (default <code>https://terpai.umd.edu</code> ).                                                                                        |
| Credentials  | Valid Nebula session values (see below).                                                                                                                                              |

### 4 Environment variables

| Variable                              | Required | Description                                                                    |
|---------------------------------------|----------|--------------------------------------------------------------------------------|
| <code>TERPAI_BEARER</code>            | Yes      | Bearer token value <b>without</b> the <b>Bearer</b> prefix (the code adds it). |
| <code>TERPAI_CONVERSATION_ID</code>   | Yes      | Conversation UUID used in the segments URL path.                               |
| <code>TERPAI_PARENT_SEGMENT_ID</code> | Yes      | Parent segment for <code>lineage.parentSegmentId</code> (Question lineage).    |
| <code>TERPAI_BASE_URL</code>          | No       | Override API host; default <code>https://terpai.umd.edu</code> .               |

If any required value is missing, `compute_terpai_scheduling_summary` returns `skipped` with a short reason instead of calling the network.

### 5 Data contract — `jupiterp_response`

The function `compute_terpai_scheduling_summary(jupiterp_response, preferences=None)` expects a **dictionary** shaped like a successful Jupiterp merge used elsewhere in ShellAI:

- **ok** (truthy): if falsey, the client sets `skipped` to "courses payload not ok" and returns without calling Terpai.
- **course\_details**: list of course dicts; each may include `course_code`, `name`, `min_credits` / `max_credits`, and `sections` (list). The client **slims** the first 20 courses and the first section per course for the prompt (meetings truncated).

Optional fields used when present: `student_level_inferred`, `course_level_policy`, `matched_codes`, `preferences.schedule_constraints` (or the same under the top-level `preferences` argument).

A **minimal** example sufficient for a dry-run:

```
{
  "ok": true,
  "course_details": [
    {
      "course_code": "CMSC216",
      "name": "Programming Computer Science",
      "min_credits": 3,
      "sections": [
        {
          "section_code": "0101",
          "open_seats": 5,
          "total_seats": 120,
          "meetings": [{"days": "MWF", "start_time": "10:00", "end_time": "10:50"}]
        }
      ]
    }
  ]
}
```

## 6 Public API (summary)

| Name                                                                               | Role                                                                                                                                    |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>terpai_configured()</code>                                                   | Returns <code>True</code> if URL, headers, and parent segment can be built from env.                                                    |
| <code>ask_terpai(question, parent_segment_id=None, timeout=120)</code>             | Low-level: POST question, consume SSE-style lines, return <code>(text, new_segment_id_or_none)</code> .                                 |
| <code>build_scheduling_question(slim)</code>                                       | Builds the full prompt string from the slimmed dict.                                                                                    |
| <code>compute_terpai_scheduling_summary(jupyter_response, preferences=None)</code> | Processes <code>jupyter_response</code> ; returns <code>{"summary", "error", "skipped"}</code> with string or <code>None</code> values. |

Return shape is typed in code as `TerpaiSchedulingResult` (`TypedDict`) for editors and static checkers.

## 7 How to run locally

From the repository root (after installing backend deps, e.g. `pip install -r backend/requirements.txt`):

Commands: **check** (env only), **dry-run** (fixture prompt, no HTTP), **live** (real API with fixture).

```
cd backend
python terpai_client.py check
python terpai_client.py dry-run
python terpai_client.py live
```

Exit codes for `check`: **0** if all required env vars are set, **1** if anything is missing (useful for scripts).  
 For `live`: **0** if a summary was returned, **1** on empty/error response, **2** if the call was skipped (missing env or payload not ok).

To import from another script (with `backend` on `PYTHONPATH` or run from `backend/`):

```
from terpai_client import compute_terpai_scheduling_summary, MINIMAL_JUPITERP_FIXTURE

result = compute_terpai_scheduling_summary(MINIMAL_JUPITERP_FIXTURE)
print(result)
```

## 8 Operational flow (text)

1. Validate env and `jupiterp_response["ok"]`.
2. `_slim_courses_for_prompt` reduces token load.
3. `build_scheduling_question` embeds JSON (capped length) plus system instructions.
4. `ask_terpai` streams response lines, base64-decodes when applicable, extracts text and optional new segment id.
5. Result dict is returned to the caller (e.g. merged into `POST /getCourses` when enabled in `main.py`).

## 9 Security and compliance

- **Treat `TERPAI_BEARER` as a secret**; never commit it to git. Use `.env` / secret manager in deployment.
- Prefer **short-lived tokens** and institutional guidance on acceptable automation against campus systems.
- This module does **not** disable TLS verification; for dev-only TLS bypass, use the separate FastAPI proxy patterns documented in the main `README.md`, not this client.

## 10 Troubleshooting

| Symptom                                     | Likely cause                                       |
|---------------------------------------------|----------------------------------------------------|
| <code>skipped</code> mentions env           | Missing or blank <code>TERPAI_*</code> variables.  |
| <code>skipped</code> courses payload not ok | <code>ok</code> is false or missing in input dict. |

| Symptom                              | Likely cause                                                           |
|--------------------------------------|------------------------------------------------------------------------|
| <code>error</code> with HTTP message | Token expired, wrong conversation/parent id, or network/TLS failure.   |
| <code>error</code> empty response    | Stream contained no extractable assistant text (API or format change). |

## 11 Related files in this repository

- `backend/terpai_chat.py` — interactive REPL and shared streaming helpers used by the HTTP proxy router.
- `backend/terpai_umd_routes.py` — FastAPI `/api/terpai` proxy for browser-driven flows.
- `backend/main.py` — `terpai_scheduling` merge is currently commented out; re-enable when tokens and error semantics are production-ready.

## 12 Revision history

| Version | Date    | Notes                                                    |
|---------|---------|----------------------------------------------------------|
| 1.0     | 2026-05 | Initial handoff PDF generated from this Markdown source. |